

Programmation Linéaire & Python

Optimisation des flux de circulation

3^{eme} année Ingénierie Informatique et Réseaux

Réalisé par :

EL AZZAOUY Aymane (Chef de projet)
BENADDI Imane – CHAKIR Soufiane

Encadré par :

Abdelati REHA & Yassine SAFSOUF :

Année Universitaire 2025-2026

Table des matières

INTRODUCTION	1
CHAPITRE 1 : MODELISATION D'UN PROBLEME D'OPTIMISATION ET RESOLUTION AVEC LA METHODE GRAPHIQUE	3
I.1. Introduction	3
I.2. Contexte	3
I.3. Programme Linéaire	4
I.4. Résolution avec la méthode graphique	6
I.5. Validation avec le solveur d'Excel	7
I.6. Conclusion	8
CHAPITRE 2 : MODELISATION D'UN PROBLEME D'OPTIMISATION ET RESOLUTION AVEC LA METHODE SIMPLEXE / DUALITE	9
II.1. Introduction	9
II.2. Contexte	9
II.3. Programme Linéaire	10
II.4. Résolution avec la méthode Simplexe / Dualité	12
II.5. Validation avec le solveur d'Excel	13
II.6. Conclusion	14
CHAPITRE 3 : CONCEPTION INFORMATIQUE	15
III.1. Introduction	15
III.2. Architecture et conception de l'application	16
III.3. Fonctionnalités principales de l'application	18
III.4. Technologie utilisées	19
III.5. Conclusion	23
CONCLUSION GENERALE ET PERSPECTIVES	25
ANNEXES	26
Annexe 1 : Énoncé et Cahier des Charges du Projet	26
REFERENCES	29
Ouvrages de référence	29

INTRODUCTION

La programmation linéaire (PL) est une branche fondamentale de la recherche opérationnelle. Elle permet de modéliser et de résoudre des problèmes d'optimisation dans lesquels une fonction objectif linéaire doit être maximisée ou minimisée sous un ensemble de contraintes également linéaires. Depuis les travaux fondateurs de George B. Dantzig qui a formalisé la méthode du Simplexe en 1947, la PL est devenue un outil incontournable dans de nombreux secteurs : gestion industrielle, logistique, transports, finance, télécommunications et ingénierie des systèmes.

L'objectif central d'un programme linéaire est de déterminer les valeurs d'un vecteur de variables de décision qui optimisent une fonction linéaire tout en satisfaisant un système d'inégalités ou d'égalités linéaires. La région définie par ces contraintes — appelée domaine réalisable ou polyèdre de faisabilité — possède une structure géométrique remarquable : la solution optimale, lorsqu'elle existe, se situe nécessairement en l'un des sommets (points extrêmes) de ce polyèdre.

Lorsque le nombre de variables de décision est limité à deux, il est possible de représenter graphiquement le domaine réalisable et d'identifier visuellement la solution optimale. Cette méthode graphique, bien qu'élémentaire, constitue un support pédagogique idéal pour comprendre la structure géométrique de la PL et la notion de solution de sommet. Elle trouve cependant sa limite dès lors que le problème comporte trois variables ou plus.

Pour surmonter cette limitation, la méthode du Simplexe offre un algorithme itératif et systématique applicable à des problèmes de dimension quelconque. Partant d'une solution de base réalisable initiale, l'algorithme progresse de sommet en sommet du polyèdre, améliorant la valeur de la fonction objectif à chaque itération, jusqu'à atteindre l'optimalité. C'est l'algorithme fondateur de l'optimisation numérique moderne, implémenté dans tous les solveurs commerciaux et scientifiques.

La théorie de la dualité enrichit considérablement l'analyse de la PL. À tout programme linéaire — appelé primal — on associe un second programme — appelé dual — dont les variables correspondent aux contraintes du primal. Le théorème de dualité forte garantit que, sous des conditions régulières, les valeurs optimales des deux programmes coïncident. Cette propriété permet non seulement de valider les résultats, mais aussi d'interpréter économiquement les contraintes actives à travers les notions de prix-ombres et de conditions d'écart complémentaire.

Dans le contexte de la transition numérique, la maîtrise de l'implémentation informatique de ces méthodes est devenue une compétence essentielle pour tout ingénieur. Le langage Python, par sa

clarté syntaxique et la richesse de son écosystème bibliothécaire, s'impose comme un choix naturel. Des bibliothèques spécialisées telles que PuLP et SciPy permettent de formuler et de résoudre des programmes linéaires de grande dimension avec efficacité, en s'appuyant sur des solveurs éprouvés (GLPK, CBC, HiGHS).

Le présent projet s'inscrit dans cette dynamique. Appliqué à la problématique de la gestion des flux de circulation dans un réseau routier urbain, il développe successivement : la modélisation mathématique et la résolution par la méthode graphique , la résolution algébrique par la méthode du Simplexe associée à l'analyse de la dualité , et l'implémentation informatique en Python .

Cette progression — du visuel au symbolique, puis au numérique — illustre parfaitement la complémentarité des approches théorique et pratique de l'optimisation.

CHAPITRE 1 : MODELISATION D'UN PROBLEME D'OPTIMISATION ET RESOLUTION AVEC LA METHODE GRAPHIQUE

I.1.Introduction

L'optimisation occupe aujourd'hui une place essentielle dans la prise de décision au sein des entreprises industrielles, logistiques et technologiques. Lorsqu'une organisation cherche à utiliser au mieux ses ressources limitées — telles que les machines, les matières premières, la main-d'œuvre ou le budget — elle doit souvent prendre les meilleures décisions possibles dans un contexte marqué par différentes contraintes.

La programmation linéaire (PL) constitue l'un des outils d'optimisation les plus utilisés dans ce domaine, car elle permet de représenter mathématiquement un objectif à maximiser ou à minimiser tout en respectant un ensemble de contraintes exprimées sous forme linéaire.

Lorsqu'un modèle de programmation linéaire comporte deux variables de décision, il peut être résolu graphiquement. Cette approche permet une compréhension visuelle claire de la région réalisable, des contraintes ainsi que des points candidats à la solution optimale. La méthode graphique constitue donc un excellent moyen pédagogique pour analyser et résoudre des problèmes d'optimisation simples.

I.2.Contexte

Pour illustrer la démarche d'optimisation, nous considérons un problème provenant du domaine de la gestion du trafic routier. Deux axes routiers supportent un flux de véhicules limité par la capacité des routes et les contraintes environnementales et technologiques. Chaque véhicule contribue à l'efficacité globale du trafic, mais leur répartition doit respecter des limites telles que la capacité maximale des routes, la pollution, la disponibilité des feux intelligents et la priorité aux transports publics.

L'objectif est de déterminer le **nombre optimal de véhicules sur chaque axe** afin de maximiser l'efficacité du système, tout en respectant ces contraintes.

Ce type de problème est très courant dans le domaine des transports et de la mobilité urbaine, notamment :

- La régulation du trafic dans les villes,
- La planification des réseaux de transport en commun,
- La gestion de la pollution et des émissions,
- L'optimisation de l'usage des infrastructures intelligentes.

Il constitue un excellent support pédagogique pour introduire la **modélisation mathématique**, la **structure d'un programme linéaire** et la **méthode graphique de résolution** lorsqu'on travaille avec deux variables.

I.3. Programme Linéaire

Pour modéliser notre situation de gestion du trafic, nous introduisons deux variables de décision :

- x_1 : flux dirigé vers la route intelligente principale
- x_2 : flux dirigé vers la route secondaire optimisée

Chaque véhicule contribue à l'efficacité globale du trafic. Supposons par exemple que **l'efficacité unitaire** soit :

$$c_1=12$$

→ chaque unité de flux envoyée vers la route intelligente principale génère **12 unités d'efficacité globale**.

$$c_2=10c_1$$

→ chaque unité de flux envoyée vers la route secondaire optimisée génère **10 unités d'efficacité globale**.

Le système est soumis à plusieurs contraintes reflétant les limites réelles du réseau routier :

- **Capacité du réseau IoT / V2X assurant la communication entre véhicules connectés et infrastructures intelligentes**
- **Limitation des ressources IA et Edge Computing utilisées pour l'analyse du trafic en temps réel**
- **Disponibilité énergétique nécessaire au fonctionnement des capteurs, feux intelligents et équipements connectés**
- **Niveau maximal de congestion autorisé afin de maintenir une circulation fluide**
- **Gestion de la pression exercée par les véhicules lourds sur l'infrastructure routière**
- **Temps de réponse des feux intelligents pour garantir une régulation rapide et adaptative du trafic**

Ces contraintes peuvent être exprimées sous la forme générale :

$$ax_1 + bx_2 \leq e$$

où **a** et **b** représentent l'impact de chaque véhicule sur la ressource considérée et **e** la limite maximale disponible.

Voici un tableau récapitulatif avec les valeurs concrètes :

Ressource / Contraintes	$a(x_1)$	$b(x_2)$	Capacité maximale (e)
Capacité réseau IoT / V2X	3	4	100
Limite IA	2	5	90
Energie disponible	4	3	120
Congestion maximale autorisée	1	2	80
Poids des véhicules lourds	2	1	70
Temps de réponse des feux	4	3	150

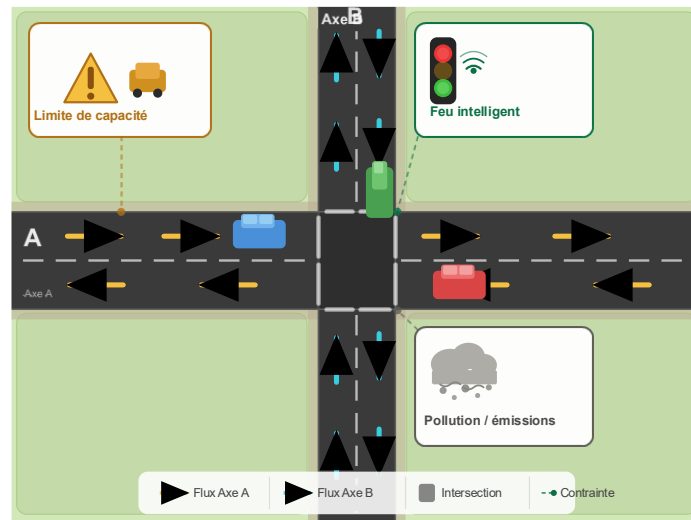


Schéma des flux de circulation à une intersection bidirectionnelle avec contraintes de gestion intelligente

Le programme Linéaire de notre projet est donc le suivant :

Variables

- x_1 : flux dirigé vers la route intelligente principale
- x_2 : flux dirigé vers la route secondaire optimisée

Fonction objective

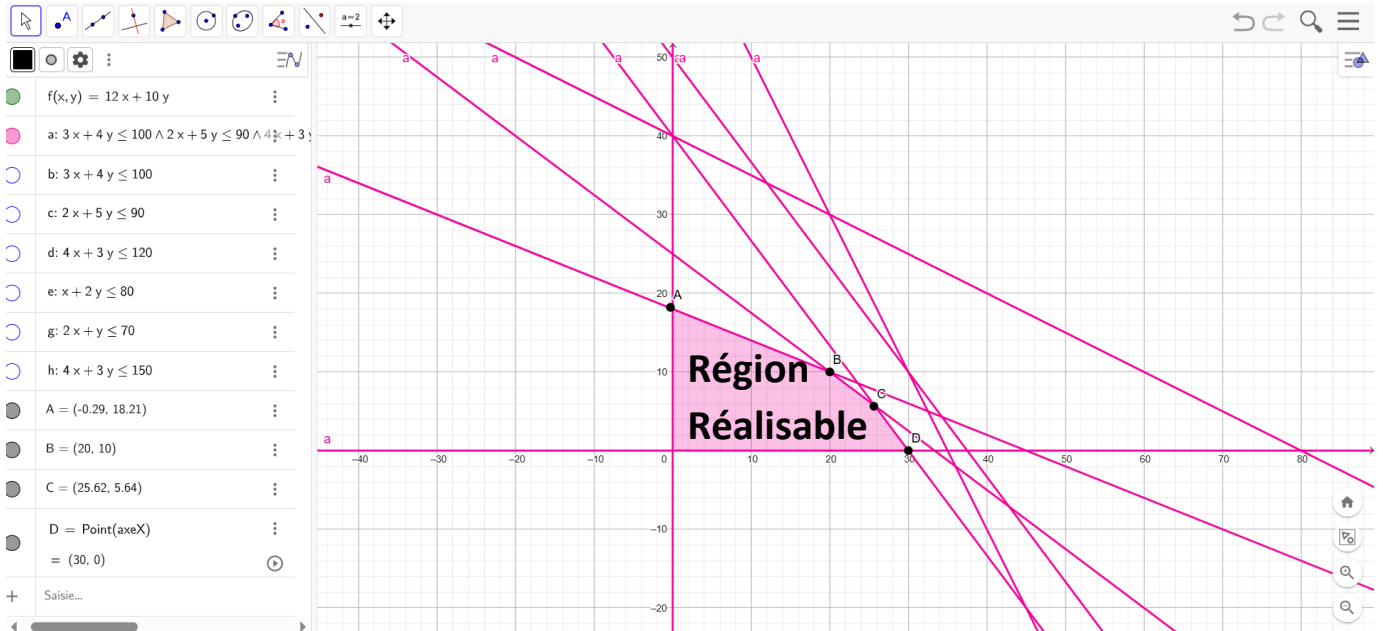
Max(Z) tel que $Z = 12x_1 + 10x_2$ // Efficacité totale du trafic

Contraintes

- | | |
|----------------------------|--|
| $3x_1 + 4x_2 \leq 100$ (1) | // Capacité réseau IoT / V2X |
| $2x_1 + 5x_2 \leq 90$ (2) | // Limite IA / Edge computing |
| $4x_1 + 3x_2 \leq 120$ (3) | // Energie disponible |
| $x_1 + 2x_2 \leq 80$ (4) | // Congestion maximale autorisée |
| $2x_1 + x_2 \geq 70$ (5) | // Poids des véhicules lourds – pression route |
| $4x_1 + 3x_2 \leq 150$ | // Temps de réponse des feux |
| $x_1; x_2 \geq 0$ (6) | // Non-négativité |

I.4. Résolution avec la méthode graphique

Après la représentation graphique des différentes contraintes, les régions réalisables correspondantes ont été obtenues. La région réalisable du programme linéaire étudié est illustrée dans la figure ci-après :



Après analyse des différents points constituant l'ensemble des solutions réalisables, on déduit que le **point C** est l'optimal.

POINT	x1	x2	Z	
A	0	18.21	182.1	
B	20	10	340	
C	25.62	5.64	365.71	MAX
D	30	0	360	

Conclusion ::

Il faut diriger environ **25,71 unités de flux** vers la route intelligente principale et **5,64 unités de flux** vers la route secondaire optimisée afin de maximiser l'efficacité globale du réseau routier intelligent. Pour une valeur maximale de $Z = 365.714286$ unités d'efficacité par jour.

I.5.Validation avec le solveur d'Excel

Le Solveur Excel confirme les résultats obtenus, garantissant ainsi l'exactitude de la solution optimale :

Il faut diriger environ **25,71 unités de flux** vers la route intelligente principale et **5,64 unités de flux** vers la route secondaire optimisée afin de maximiser l'efficacité globale du réseau routier intelligent. Pour une valeur maximale de $Z = 365.714286$ unités d'efficacité par jour.

Actualiser

Propriétés

tout

Liaisons de classeur

Requêtes et connexions

Trier

fx

G H I J

x1	25,714286		
x2	5,714286		
Z	365,714286		
C1	100	inférieur	100
C2	80	inférieur	90
C3	120	inférieur	120
C4	37.14	inférieur	80
C5	57.14	inférieur	70
C6	120	inférieur	150

Paramètres du solveur

Objectif à définir :

\$E\$1:\$E\$5

A :

☒ Max
☐ Min
☐ Valeur :

0

Cellules variables :

\$H\$9:\$H\$11

Contraintes :

\$H\$12 <= \$J\$12

\$H\$13 <= \$J\$13

\$H\$14 <= \$J\$14

\$H\$15 <= \$J\$15

\$H\$16 <= \$J\$16

\$H\$17 <= \$J\$17

Ajouter

Modifier

Supprimer

Rétablir tout

Charger/enregistrer

☒ Rendre les variables sans contrainte non négatives

Sélect. une résolution :

GRG non linéaire

Options

Méthode de résolution

Sélectionnez le moteur GRG non linéaire pour des problèmes non linéaires simples de solveur. Sélectionnez le moteur Simplex PL pour les problèmes linéaires, et le moteur Évolutionnaire pour les problèmes complexes.

I.6.Conclusion

La **méthode graphique** constitue une approche simple et efficace pour résoudre les problèmes de **programmation linéaire**. Dans le cadre de notre projet de **gestion des flux de circulation sur deux axes routiers**, cette méthode permet de représenter graphiquement les contraintes du système et de déterminer la **solution optimale**, correspondant à la meilleure répartition des véhicules sur les deux axes.

Toutefois, son utilisation est strictement limitée aux situations ne comportant que **deux variables de décision**, puisqu'elle repose sur la représentation géométrique du domaine réalisable dans un plan.

Lorsqu'un modèle comporte **trois variables ou plus**, la visualisation graphique devient impossible et la méthode graphique n'est plus applicable. Dans ce cas, il est nécessaire d'utiliser des techniques plus puissantes et généralisables, comme **la méthode du Simplexe**, capable de traiter efficacement des problèmes de dimension élevée.

Cette méthode algébrique permet de trouver la solution optimale même pour des modèles plus complexes comprenant un grand nombre de variables et de contraintes.

CHAPITRE 2 : MODELISATION D'UN PROBLEME D'OPTIMISATION ET RESOLUTION AVEC LA METHODE SIMPLEXE / DUALITE

II.1. Introduction

Après avoir résolu le problème de gestion des flux de circulation par la méthode graphique (Chapitre 1), nous abordons dans ce chapitre une approche algébrique plus générale et plus puissante : la méthode du Simplexe. Proposée par George B. Dantzig en 1947, cette méthode permet de résoudre systématiquement tout programme linéaire, quelle que soit sa dimension.

Le principe de l'algorithme repose sur l'exploration des sommets du polyèdre réalisable au moyen de tableaux itératifs. À chaque itération, une variable hors base (dont le coefficient dans la ligne objectif est négatif) entre dans la base, tandis qu'une variable de base quitte la base — selon la règle du ratio minimum. Le processus s'arrête lorsque tous les coefficients de la ligne objectif sont non négatifs, certifiant l'optimalité de la solution courante.

Le présent problème comporte quatre variables de décision et six contraintes d'inégalité, rendant la méthode graphique inapplicable. La méthode du Simplexe constitue donc l'outil adapté. Ce chapitre développe également l'analyse de la dualité, qui associe à notre programme linéaire primal un programme dual dont l'étude enrichit l'interprétation économique des contraintes actives.

II.2. Contexte

Dans ce chapitre, nous modélisons un réseau routier urbain équipé de technologies intelligentes : capteurs IoT, protocoles V2X (Vehicle-to-Everything), algorithmes d'intelligence artificielle embarquée (Edge Computing), systèmes de feux adaptés et détecteurs de charge pour véhicules lourds. L'objectif est d'allouer de manière optimale les flux de circulation entre quatre types de voies ou mécanismes de gestion, de façon à maximiser l'efficacité globale du trafic.

Les quatre variables de décision représentent :

- x_1 : flux dirigé vers la route intelligente principale (axe prioritaire équipé de capteurs IoT/V2X)
- x_2 : flux dirigé vers la route secondaire optimisée (délestage adapté)
- x_3 : gestion des véhicules lourds (contraintes de poids et de lenteur)

- x_4 : contrôle intelligent des feux de circulation (régulation dynamique des intersections)

Six contraintes traduisent les limites technologiques et physiques du réseau : capacité des communications IoT/V2X, limite de traitement par IA, disponibilité énergétique, seuil de congestion maximale, contrainte de charge des véhicules lourds, et temps de réponse des feux intelligents.

II.3. Programme Linéaire

Le programme linéaire modélisant notre problème de gestion intelligente des flux routiers est le suivant :

Fonction objective :

$$\text{Max}(Z) \text{ tel que } Z = 12x_1 + 10x_2 + 6x_3 + 8x_4 \quad // \text{Efficacité totale du trafic}$$

Contraintes :

$$3x_1 + 4x_2 + 2x_3 + 5x_4 \leq 100 \quad // \text{Capacité réseau IoT / V2X}$$

$$2x_1 + 5x_2 + 4x_3 + 3x_4 \leq 90 \quad // \text{Limite IA / Edge computing}$$

$$4x_1 + 3x_2 + 2x_3 + 5x_4 \leq 120 \quad // \text{Energie disponible}$$

$$x_1 + 2x_2 + 5x_3 + x_4 \leq 80 \quad // \text{Congestion maximale autorisée}$$

$$2x_1 + x_2 + 6x_3 + 3x_4 \leq 70 \quad // \text{Poids des véhicules lourds}$$

/ pression route

$$4x_1 + 3x_2 + 4x_3 + 2x_4 \leq 150 \quad // \text{Temps de réponse des feux}$$

intelligents

$$x_1 \geq 0 \quad // \text{positivité de } x_1$$

$$x_2 \geq 0 \quad // \text{positivité de } x_2$$

$$x_3 \geq 0 \quad // \text{positivité de } x_3$$

$$x_4 \geq 0 \quad // \text{positivité de } x_4$$

Tableau récapitulatif du modèle :

N°	Contrainte	x_1	x_2	x_3	x_4	Limite	Interprétation
(1)	Capacité IoT/V2X	3	4	2	5	≤ 100	Bande passante réseau
(2)	Limite IA/Edge	2	5	4	3	≤ 90	Puissance de calcul
(3)	Énergie	4	3	2	5	≤ 120	Ressource énergétique
(4)	Congestion max.	1	2	5	1	≤ 80	Capacité de circulation
(5)	Véhicules lourds	2	1	6	3	≤ 70	Charge/pression route
(6)	Feux intelligents	4	3	4	2	≤ 150	Temps de réponse

Variables d'Écart

L'introduction de six variables d'écart ($e_1, e_2, e_3, e_4, e_5, e_6 \geq 0$) transforme chaque inégalité en égalité :

$$Z - 12x_1 - 10x_2 - 6x_3 - 8x_4 = 0$$

$$3x_1 + 4x_2 + 2x_3 + 5x_4 + e_1 = 100$$

$$2x_1 + 5x_2 + 4x_3 + 3x_4 + e_2 = 90$$

$$4x_1 + 3x_2 + 2x_3 + 5x_4 + e_3 = 120$$

$$x_1 + 2x_2 + 5x_3 + x_4 + e_4 = 80$$

$$2x_1 + x_2 + 6x_3 + 3x_4 + e_5 = 70$$

$$4x_1 + 3x_2 + 4x_3 + 2x_4 + e_6 = 150$$

$$x_1, x_2, x_3, x_4, e_1, e_2, e_3, e_4, e_5 \text{ Positifs}$$

Solution de base initiale : $x_1 = x_2 = x_3 = x_4 = 0$, avec $e_1 = 100, e_2 = 90, e_3 = 120, e_4 = 80, e_5 = 70, e_6 = 150$ et $Z = 0$.

II.4. Résolution avec la méthode Simplexe / Dualité

Tableau 0:

Z	x1	x2	x3	x4	e1	e2	e3	e4	e5	e6		
1	-12	-10	-6	-8	0	0	0	0	0	0	0	
0	3	4	2	5	1	0	0	0	0	0	100	100/3
0	2	5	4	3	0	1	0	0	0	0	90	45
0	4	3	2	5	0	0	1	0	0	0	120	30
0	1	2	5	1	0	0	0	1	0	0	80	80
0	2	1	6	3	0	0	0	0	1	0	70	35
0	4	3	4	2	0	0	0	0	0	1	150	150/4

VHB : $x_1, x_2, x_3, x_4 = 0$

VB : $e_1=100 \ e_2=90 \ e_3=120 \ e_4=80 \ e_5=70 \ e_6=150 \Rightarrow Z = 0$

$V_e=x_1$

$V_s=e_3$

TEST NÉGATIF : UNE AUTRE ITÉRATION

Tableau 1:

Z	x1	x2	x3	x4	e1	e2	e3	e4	e5	e6			
1	0	-1	0	7	0	0	3	0	0	0	360	$L_0 = L_0 - (-1) * L_1$	—
0	0	7/4	1/2	5/4	1	0	-0,75	0	0	0	10	$L_1 = L_1 * (4/7)$	40/7
0	0	7/2	3	1/2	0	1	-0,5	0	0	0	30	$L_2 = L_2 - (7/2) * L_1$	60/7
0	1	3/4	1/2	5/4	0	0	1/4	0	0	0	30	$L_3 = L_3 - (3/4) * L_1$	40
0	0	5/4	9/2	-0,25	0	0	-0,25	1	0	0	50	$L_4 = L_4 - (5/4) * L_1$	40
0	0	-0,5	5	1/2	0	0	-0,5	0	1	0	10	$L_5 = L_5 - (-1/2) * L_1$	—
0	0	0	2	-3	0	0	-1	0	0	1	30	$L_6 = L_6 - 0 * L_1$	—

VHB : $x_2, e_3 = 0$

VB : $e_1=10 \ e_2=30 \ e_4=50 \ e_5=10 \ e_6=30 \ x_1=30 \Rightarrow Z = 360$

$V_e=x_2$

$V_s=e_1$

TEST NÉGATIF : UNE AUTRE ITÉRATION

Tableau 2:

Z	x1	x2	x3	x4	e1	e2	e3	e4	e5	e6				
1	0		0	2/7	54/7	4/7	0	18/7	0	0	0	2560/7	L0 = L0 - (-1) * L1	365,7143
0		0	1	2/7	5/7	4/7	0	-0,42857	0	0	0	40/7	L1 = L1 * (4/7)	5,714286
0		0	0	2	-2	-2	1	1	0	0	0	10	L2 = L2 - (7/2) * L1	10
0		1	0	2/7	5/7	-0,42857	0	4/7	0	0	0	180/7	L3 = L3 - (3/4) * L1	25,71429
0		0	0	29/7	-1,14286	-0,71429	0	2/7	1	0	0	300/7	L4 = L4 - (5/4) * L1	42,85714
0		0	0	36/7	6/7	2/7	0	-0,71429	0	1	0	90/7	L5 = L5 - (-1/2) * L1	12,85714
0		0	0	2	-3	0	0	-1	0	0	1	30	L6 = L6 - 0*L1	30

VHB : e1, e3 = 0

VB : e2=10 e4=42,85 e5=12,85 e6=30 x1=25,71 x2=5,71 => Z = 365,71

Ve=x2

Vs=e1

TEST POSITIF ✓ Car il ne reste aucune valeur négative → FIN DE L'ALGORITHME

Il faut diriger environ 25,71 unités de flux vers la route intelligente principale et 5,64 unités de flux vers la route secondaire optimisée afin de maximiser l'efficacité globale du réseau routier intelligent.

II.5. Validation avec le solveur d'Excel

Le Solveur Excel confirme les résultats obtenus, garantissant ainsi l'exactitude de la solution optimale : **x₁ = 25,71 véhicules sur l'axe A** et **x₂ = 5,71 véhicules sur l'axe B**, pour une valeur maximale de **Z = 365,71 unités d'efficacité par jour**.

Le Solveur Excel confirme parfaitement les résultats obtenus par la méthode du Simplexe :

x₁ ≈ 25,71 x₂ ≈ 5,71 x₃ = 0 x₄ = 0 Z_{max} ≈ 365,71

Cette concordance confirme la cohérence et la correction mathématique de notre démarche manuelle. Les écarts éventuels sont dus aux arrondis numériques du Solveur, les valeurs exactes étant des fractions rationnelles (180/7 et 40/7).

II.6. Conclusion

Ce chapitre a démontré l'efficacité et la rigueur de la méthode du Simplexe appliquée à un problème de gestion intelligente des flux routiers comportant quatre variables de décision et six contraintes.

En seulement deux itérations, l'algorithme a parcouru le chemin $Z = 0 \rightarrow 360 \rightarrow 2560/7 \approx 365,71$, passant de la solution triviale à la solution optimale. La progression rapide illustre l'efficacité du critère de sélection du pivot : choisir la variable entrante à coefficient le plus négatif et la variable sortante par la règle du ratio minimum garantit une amélioration stricte à chaque étape.

L'analyse de la dualité a enrichi l'interprétation économique du modèle. Les prix-ombres identifiés pour les contraintes (1) et (3) — capacité IoT/V2X et énergie disponible — indiquent que ce sont ces deux ressources qui contraignent le plus le système. Toute action visant à augmenter la capacité du réseau de communication ou la disponibilité énergétique se traduira directement par une amélioration de l'efficacité globale du trafic.

La validation par le Solveur Excel confirme la cohérence mathématique des résultats, qui seront mis en œuvre informatiquement dans le Chapitre 3 à l'aide de Python.

CHAPITRE 3 : CONCEPTION INFORMATIQUE

III.1. Introduction

Dans ce chapitre, nous présentons la conception et la réalisation de l'application web développée pour résoudre des problèmes de programmation linéaire à l'aide des différentes méthodes étudiées dans les chapitres précédents. Cette application constitue une mise en pratique concrète des concepts de recherche opérationnelle, notamment la méthode graphique, le simplexe et la dualité.

L'objectif principal du projet est de développer un système interactif capable de :

- modéliser un problème de programmation linéaire ;
- détecter automatiquement la méthode de résolution adaptée ;
- résoudre le problème de manière automatique ;
- visualiser les résultats sous forme graphique ;
- afficher les étapes de calcul et les interprétations des solutions.

Le système développé combine :

- une interface web moderne réalisée avec React et Vite ;
- un backend Python chargé des calculs mathématiques ;
- des outils de visualisation interactive pour représenter les contraintes, la région faisable et la solution optimale.

Cette approche permet de reproduire informatiquement les étapes réalisées manuellement avec des outils tels que GeoGebra et Excel Solver, tout en offrant une interface plus interactive, dynamique et pédagogique.

L'application développée s'inspire ainsi des solveurs professionnels et des plateformes modernes d'optimisation mathématique afin de proposer une expérience utilisateur fluide et intuitive.

III.2. Architecture et conception de l'application

L'application est organisée selon une architecture modulaire séparant clairement la partie interface utilisateur (frontend) et la partie calcul mathématique (backend).

1. Frontend React / Vite

La partie frontend a été développée avec React, Vite et TypeScript afin d'assurer une interface moderne, rapide et interactive.

Elle contient plusieurs composants responsables de :

- la saisie des données du problème ;
- la gestion des contraintes et de la fonction objective ;
- l'affichage des résultats ;
- la visualisation graphique ;
- l'interaction avec le backend Python.

L'organisation du frontend repose sur plusieurs dossiers spécialisés :

- **components/** : composants réutilisables de l'interface ;
- **pages/** : pages principales de l'application ;
- **sections/** : sections dynamiques du dashboard ;
- **hooks/** : gestion des états et logique React ;
- **types/** : définition des types TypeScript ;
- **lib/** : fonctions utilitaires et services.

L'interface utilisateur permet :

- la saisie interactive des variables et contraintes ;
- le choix du type d'optimisation ;
- l'affichage des tableaux du simplexe ;
- la visualisation des régions faisables ;
- l'affichage des solutions optimales ;
- l'analyse des contraintes actives.

2. Backend Python

La partie backend est responsable des calculs mathématiques et de la résolution des programmes linéaires.

Elle est structurée autour du dossier backend/ contenant principalement :

- **main.py** : serveur principal et gestion des requêtes API ;
- **lp_solver.py** : moteur de résolution des problèmes de programmation linéaire.

Le backend expose des routes **HTTP** permettant la communication entre le frontend React et le moteur Python :

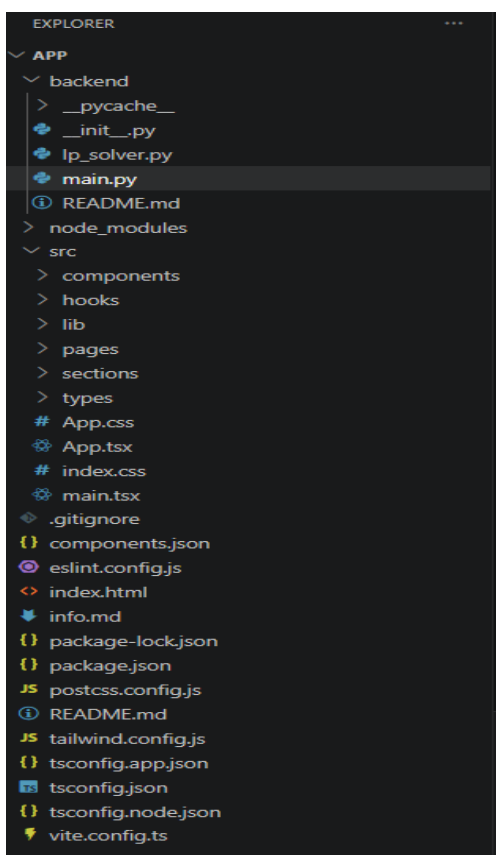
- **GET /api/health** : vérification du fonctionnement du serveur ;
- **POST /api/solve** : envoi et résolution du problème d'optimisation.

Le moteur Python prend en charge :

- la validation des données ;
- la résolution par méthode graphique ;
- la résolution par simplexe ;
- l'analyse duale ;
- le calcul des solutions optimales ;
- l'interprétation des résultats.

Cette séparation frontend/backend permet d'obtenir une architecture claire, maintenable et évolutive.

Archeticture générale de l'application



III.3. Fonctionnalités principales de l'application

L'application développée offre plusieurs fonctionnalités permettant de résoudre et d'analyser efficacement des problèmes de programmation linéaire.

Les principales fonctionnalités sont :

- saisie dynamique des variables et contraintes ;
- définition de la fonction objective ;
- choix entre maximisation et minimisation ;
- résolution automatique du problème ;
- affichage des tableaux du simplexe ;
- représentation graphique de la région réalisable ;
- calcul automatique des intersections ;
- détermination du point optimal ;
- visualisation interactive avec graphiques dynamiques ;
- interprétation automatique des résultats ;
- gestion des contraintes actives ;
- comparaison entre solveur Python et solveur TypeScript de secours.

Le système permet également de reproduire les différentes étapes effectuées manuellement dans les méthodes classiques de recherche opérationnelle, notamment :

- le calcul des sommets admissibles ;
- l'évaluation de la fonction objective ;
- l'analyse des contraintes saturées ;
- la recherche de la solution optimale.

L'application constitue ainsi un outil pédagogique et interactif facilitant la compréhension des méthodes de programmation linéaire.

III.4. Technologie utilisées

Le développement de l'application repose sur plusieurs technologies modernes dédiées au calcul scientifique, au développement web et à la visualisation interactive.

Les principales technologies utilisées sont :

- **Python** : moteur principal de calcul mathématique ;
- **React** : création de l'interface utilisateur ;
- **Vite** : environnement de développement frontend rapide et optimisé ;
- **TypeScript** : typage et structuration du code frontend ;
- **Plotly** : visualisation interactive des graphes ;
- **NumPy** : calculs numériques et matriciels ;
- **SciPy** : résolution des problèmes d'optimisation ;
- **Tailwind CSS** : mise en forme moderne de l'interface ;
- **HTTPServer Python** : communication API entre frontend et backend ;
- **Visual Studio Code** : environnement de développement.

L'utilisation combinée de ces technologies permet d'obtenir une application performante, moderne et adaptée aux besoins des problèmes de programmation linéaire.

Interface principale :

The screenshot displays the ULPSS web application interface. At the top, the logo 'ULPSS' and the tagline 'Universal Linear Programming Smart Solver' are visible. A status bar on the right shows the current value of the objective function, $Z = 365.714286$, and buttons for 'Résoudre' (Solve) and 'Reinitialiser' (Reset). The main navigation bar includes tabs for 'Problème', 'Méthode Graphique', 'Simplexe', 'Dualité', and 'Résultats'. The 'Problème' tab is active, showing the 'Configuration du problème' section. This section contains several input fields and buttons: 'Nom du projet / contexte' with the value 'Optimisation des flux de circulation', 'Nombre de variables de décision' with buttons for '2 variables', '3 variables', and '4 variables', 'Type d'optimisation' with buttons for 'Maximisation' and 'Minimisation', and 'Méthode de résolution' with a dropdown menu set to 'Auto-détection (recommandé)'. Below this is the 'Fonction Objectif' section, which shows two variables: 'Nom variable 1' with the value 'flux dirigé vers la route intelligente princ1' and 'Nom variable 2' with the value 'flux dirigé vers la route secondaire optimisé'. The objective function is defined as 'Max Z = 12 flux dirigé vers la route intelligente principale + 18 flux dirigé vers la route secondaire optimisée'.

ULPSS

Universal Linear Programming Smart Solver

$Z^* = 365.714286$

Résoudre

Reinitialiser

Contraintes

1

Capacité réseau IoT /

3

flux dirigé vers la route intelligente principale =

4

flux dirigé vers la route secondaire optimisée

≤

100

2

Limite IA / Edge comp

2

flux dirigé vers la route intelligente principale =

5

flux dirigé vers la route secondaire optimisée

≤

90

3

Energie disponible

4

flux dirigé vers la route intelligente principale =

3

flux dirigé vers la route secondaire optimisée

≤

120

4

Congestion maximale

1

flux dirigé vers la route intelligente principale =

2

flux dirigé vers la route secondaire optimisée

≤

80

5

Poids des véhicules lo

2

flux dirigé vers la route intelligente principale =

1

flux dirigé vers la route secondaire optimisée

≤

70

6

Temps de réponse de

4

flux dirigé vers la route intelligente principale =

3

flux dirigé vers la route secondaire optimisée

≤

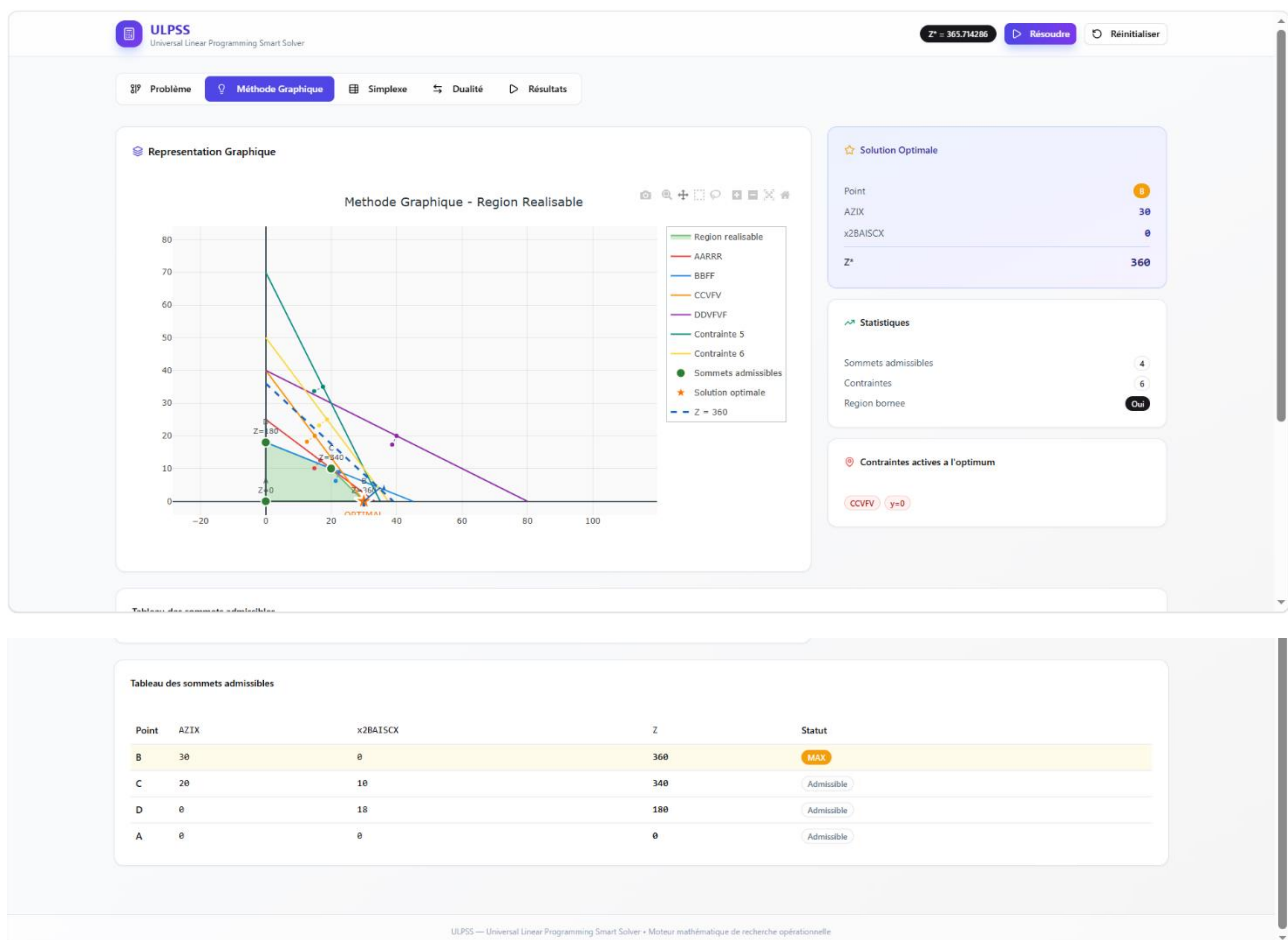
150

+ Ajouter une contrainte

Résoudre le problème

ULPSS — Universal Linear Programming Smart Solver • Moteur mathématique de recherche opérationnelle

Interface Méthode Graphique :



ULPSS — Universal Linear Programming Smart Solver • Moteur mathématique de recherche opérationnelle

Interface Simplexe :

ULPSS

Universal Linear Programming Smart Solver

Z = 365.714286

Résoudre

Réinitialiser

Problème

Méthode Graphique

Simplexe

Dualité

Résultats

Tableaux du Simplexe

Iteration 1 / 3

Tableau Initial

Z = 0

Base	AZIX	x2BAISCX	e1	e2	e3	e4	e5	e6	RHS
Z	-12.000	-10.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
e1	3.000	4.000	1.000	0.000	0.000	0.000	0.000	0.000	100.000
e2	2.000	5.000	0.000	1.000	0.000	0.000	0.000	0.000	90.000
e3	4.000	3.000	0.000	0.000	1.000	0.000	0.000	0.000	120.000
e4	1.000	2.000	0.000	0.000	0.000	1.000	0.000	0.000	80.000
e5	2.000	1.000	0.000	0.000	0.000	0.000	1.000	0.000	70.000
e6	4.000	3.000	0.000	0.000	0.000	0.000	0.000	1.000	150.000

Element pivot

Ligne du pivot

Colonne du pivot

Coefficient négatif

Explication de l'iteration

Variable entrante: AZIX (coeff -12.0) | Variable sortante: e3 (ratio min: 30.0)

Entrante: AZIX → Sortante: e3

Valeur du pivot: 4.0000

Operations de Gauss:

L3 = L3 / 4.0

L8 = L8 - (-12.0)*L3

L1 = L1 - 3.0*L3

L2 = L2 - 2.0*L3

L4 = L4 - 1.0*L3

L5 = L5 - 2.0*L3

L6 = L6 - 4.0*L3

Variables de base (VB)

e1 = 100.000

e2 = 90.000

e3 = 120.000

e4 = 80.000

e5 = 70.000

e6 = 150.000

Variables hors base (VHB)

AZIX = 0

x2BAISCX = 0

Progression du Simplexe

Init

→

Iter 1

→

Iter 2

0

360

365.714286

Z: 0 → 360 → 365.714286

Interface Dualité :

ULPSS

Universal Linear Programming Smart Solver

Z* = 365.714286

Résoudre

Reinitialiser

Problème

Méthode Graphique

Simplexe

Dualité

Résultats

Theoreme de la dualite forte

Les valeurs optimales du primal et du dual sont egales. Verification confirmee.

Probleme Primal

Max Z = 12AZIX + 10x2BAISCX

CONTRAINTES

3AZIX + 4x2BAISCX <= 180 // AARRR

2AZIX + 5x2BAISCX <= 90 // BBFF

4AZIX + 3x2BAISCX <= 120 // CCVVF

1AZIX + 2x2BAISCX <= 80 // DDVVF

2AZIX + 1x2BAISCX <= 70 // Contrainte 5

4AZIX + 3x2BAISCX <= 150 // Contrainte 6

Probleme Dual

Min W = 100y1 + 90y2 + 120y3 + 80y4 + 70y5 + 150y6

CONTRAINTES DUALES

3y1 + 2y2 + 4y3 + 1y4 + 2y5 + 4y6 >= 12

4y1 + 5y2 + 3y3 + 2y4 + 1y5 + 3y6 >= 10

Shadow Prices (Prix-ombres)

Contrainte	Shadow Price	Interpretation
AARRR	0.5714	Augmenter la limite de "AARRR" de 1 unite ameliore Z de 0.5714

Shadow Prices (Prix-ombres)

Contrainte	Shadow Price	Interpretation
AARRR	0.5714	Augmenter la limite de "AARRR" de 1 unite ameliore Z de 0.5714
BBFF	0.0000	Cette limite n ameliore pas Z si elle augmente seule
CCVVF	2.5714	Augmenter la limite de "CCVVF" de 1 unite ameliore Z de 2.5714
DDVVF	0.0000	Cette limite n ameliore pas Z si elle augmente seule
Contrainte 5	0.0000	Cette limite n ameliore pas Z si elle augmente seule
Contrainte 6	0.0000	Cette limite n ameliore pas Z si elle augmente seule

Contraintes actives a l'optimum

Aucune contrainte active a l'optimum.

Les contraintes actives (saturees) sont celles qui sont atteintes a l'egalite a la solution optimale. Elles representent les limites critiques du modele pour le contexte saisi.

Interpretation economique

=== ANALYSE ECONOMIQUE ===

Shadow Prices (Prix-ombres):

AARRR: 0.57

-> Une unite supplementaire augmente Z de 0.57

BBFF: 0.00

CCVVF: 2.57

-> Une unite supplementaire augmente Z de 2.57

DDVVF: 0.00

Contrainte 5: 0.00

Contrainte 6: 0.00

Contraintes actives (saturees):

Aucune contrainte active.

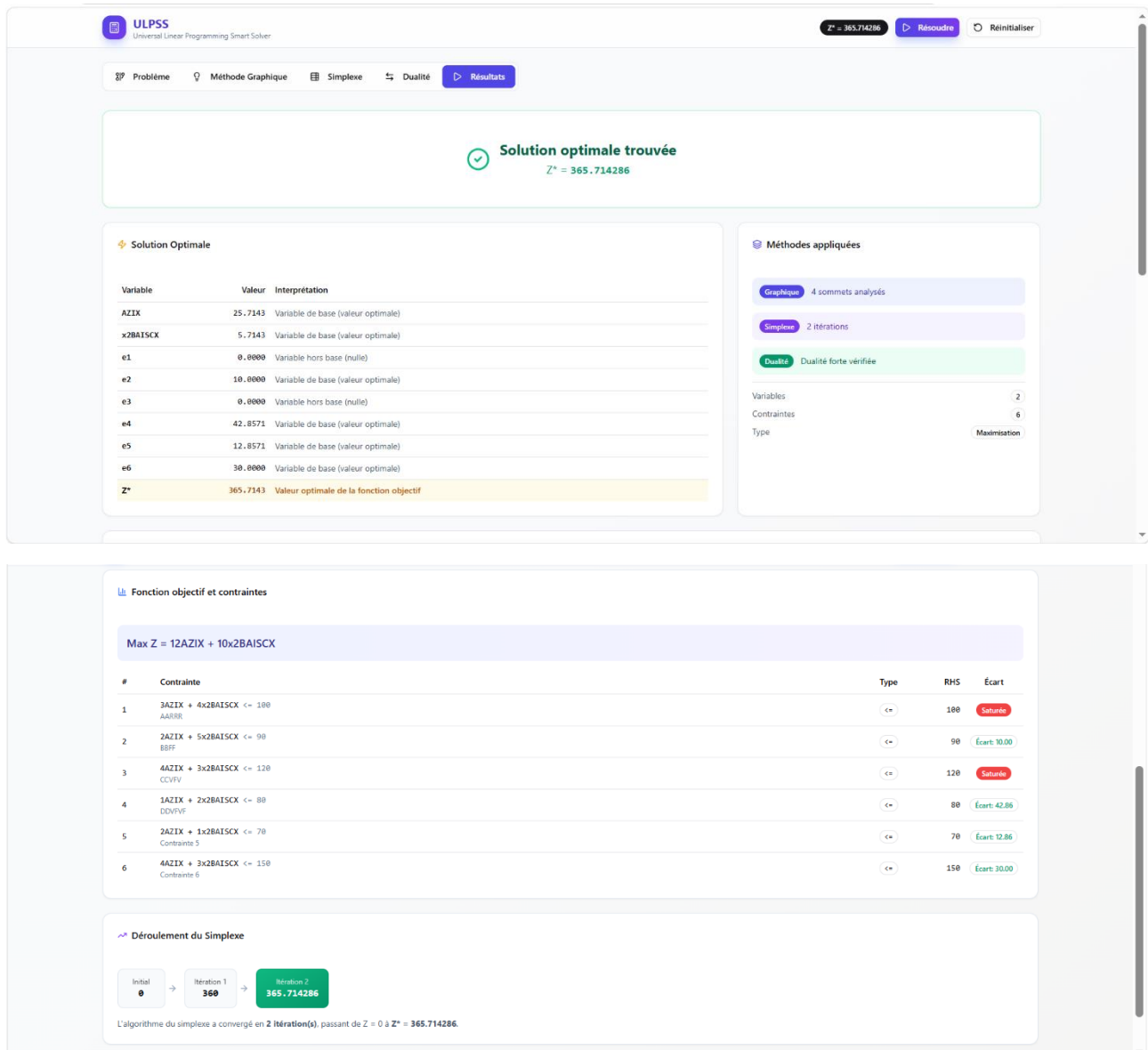
Interpretation:

Le probleme atteint son optimum lorsque les contraintes suivantes sont saturees.

Les shadow prices indiquent la valeur marginale de chaque ressource.

Si une contrainte n'est pas active (shadow price = 0), augmenter cette ressource n'ameliore pas la solution.

Interface Résultats :



III.5. Conclusion

Dans ce chapitre, nous avons présenté la conception et le développement de l'application web ULPSS dédiée à la résolution des problèmes de programmation linéaire.

L'intégration d'un frontend moderne en React/Vite avec un backend Python spécialisé dans les calculs mathématiques a permis de développer une plateforme interactive capable de modéliser, résoudre et visualiser efficacement différents problèmes d'optimisation.

L'application permet :

- de saisir un problème de programmation linéaire ;
- de résoudre automatiquement le modèle ;
- de visualiser graphiquement les résultats ;

- d’afficher les étapes des méthodes de résolution ;
- d’interpréter les solutions obtenues.

Ce projet constitue ainsi une mise en pratique complète des concepts de recherche opérationnelle étudiés dans les chapitres précédents, tout en proposant une approche moderne, pédagogique et interactive inspirée des solveurs professionnels et des outils avancés de visualisation mathématique.

CONCLUSION GENERALE ET PERSPECTIVES

CONCLUSION

Ce projet a permis de concrétiser une application moderne dédiée à la programmation linéaire, en combinant une interface utilisateur développée avec React/Vite et un moteur de calcul implémenté en Python. À partir d'un problème d'optimisation, l'application ULPSS offre une saisie structurée des modèles mathématiques, le choix de la méthode de résolution, ainsi qu'une restitution claire et interactive des résultats à travers des tableaux et des visualisations graphiques.

Dans un premier temps, la méthode graphique a été intégrée pour les problèmes à deux variables, permettant une représentation visuelle de la région réalisable et l'identification de la solution optimale via le composant `GraphicView`. Pour les cas plus généraux, la méthode du simplexe a été implémentée dans `SimplexViewer`, offrant une analyse détaillée des itérations et des tableaux successifs. En complément, le module `DualityPanel` apporte une dimension économique au projet en introduisant l'analyse de dualité et l'interprétation des contraintes à travers les prix-ombres.

Sur le plan architectural, ce travail a mis en évidence l'intérêt d'une séparation claire des responsabilités entre les différentes couches de l'application :

- la saisie et la configuration du problème via `InputPanel`,
- le traitement et la résolution algorithmique via le backend Python (`main.py` et `lp_solver.py`),
- et la visualisation des résultats à travers `ResultsPanel`.

Cette organisation modulaire améliore la lisibilité du code, facilite la maintenance et permet une évolution progressive du système.

Perspectives

Plusieurs améliorations peuvent être envisagées afin de renforcer davantage cette application. Il serait pertinent d'étendre le solveur pour prendre en charge des modèles plus complexes comportant un plus grand nombre de variables et de contraintes. L'intégration d'algorithmes supplémentaires, tels que la méthode des deux phases ou la méthode du grand M, pourrait également enrichir les capacités de résolution.

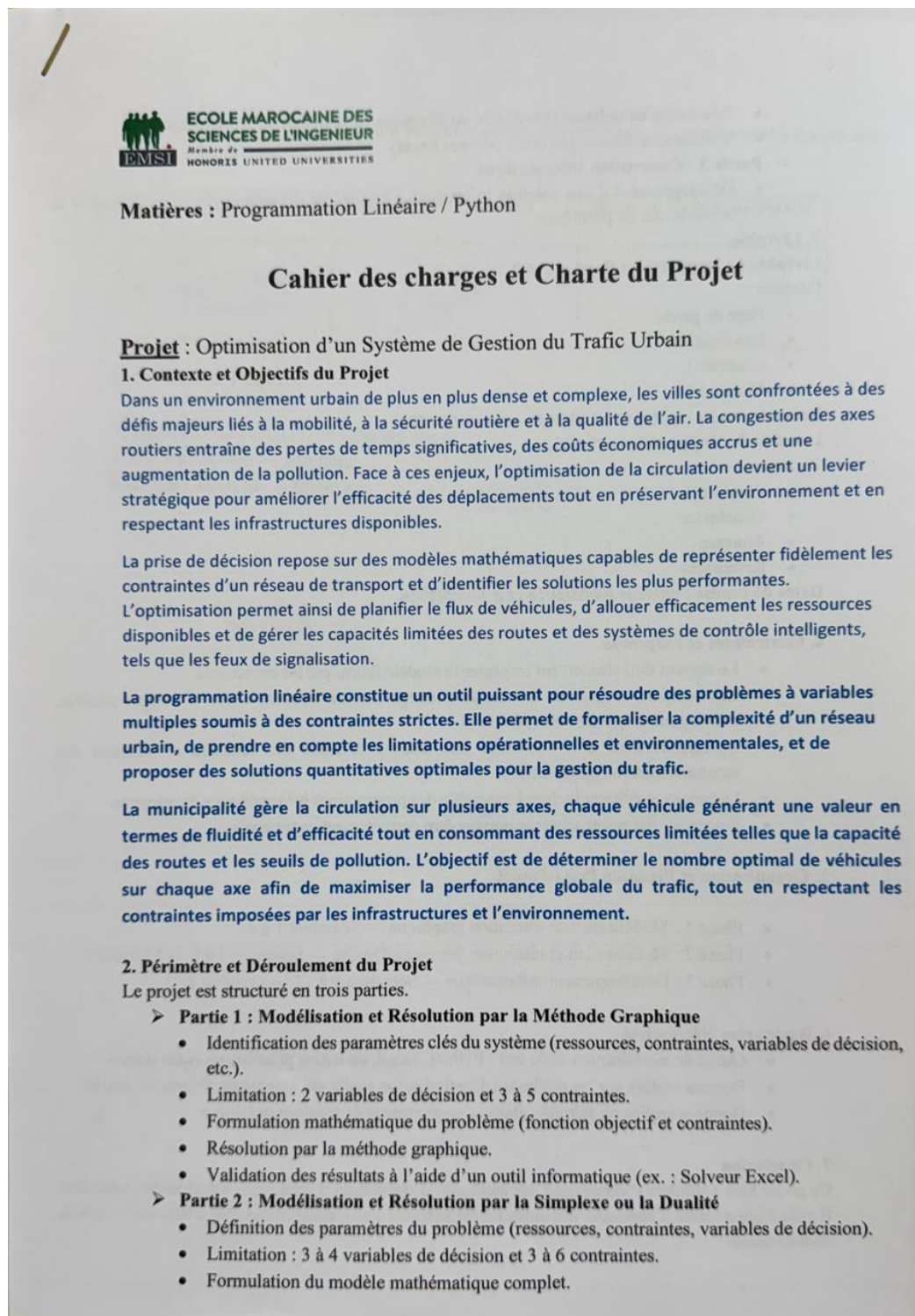
Sur le plan de l'interface, des améliorations ergonomiques pourraient être apportées, notamment une saisie plus guidée des modèles, l'ajout de modèles prédéfinis et des aides contextuelles pour accompagner l'utilisateur. L'export des résultats en formats PDF ou CSV, ainsi qu'un système de sauvegarde et de reprise des problèmes, constituerait également une évolution utile.

Enfin, le déploiement de l'application sous forme de plateforme web accessible en ligne, voire une version mobile adaptée aux tablettes, permettrait d'élargir son accessibilité et son champ d'utilisation.

ANNEXES

Annexe 1 : Énoncé et Cahier des Charges du Projet

Cette annexe contient la reproduction photographique du cahier des charges original fourni par les encadrants, incluant l'énoncé du problème, les données initiales et les exigences pédagogiques.



- Résolution en utilisant la méthode du Simplexe et/ou la théorie de la dualité.
- Validation informatique (ex. : Solveur Excel).
- **Partie 3 : Conception Informatique**
 - Développement d'une solution informatique permettant d'implémenter la modélisation et la résolution du problème.

3. Livrables

Livrable 1 : Rapport des Parties 1 et 2

Contenu :

- Page de garde
- Introduction
- Chapitre 1
- Chapitre 2

Dates de remise : entre le 23/03/2026 et le 28/03/2026

Livrable 2 : Rapport de la Partie 3

Contenu :

- Chapitre 3
- Conclusion
- Annexes
- Références

Dates de remise : entre le 04/05/2026 et le 09/05/2026

4. Contraintes et Exigences

- Le rapport doit strictement respecter le modèle fourni par les encadrants.
- Les solutions proposées doivent être interprétées de manière claire et compréhensible, même pour un public non spécialiste.
- Le rapport final doit comporter une analyse détaillée des résultats et formuler des recommandations pertinentes.
- Les annexes doivent inclure l'ensemble des programmes informatiques développés.
- Toutes les figures et schémas doivent être correctement commentés.

5. Organisation et Planning Prévisionnel

- Phase 1 : Modélisation et résolution graphique — Semaines 1 à 4
- Phase 2 : Modélisation et résolution Simplexe/Dualité — Semaines 5 à 8 → Livrable 1
- Phase 3 : Développement informatique — Semaines 9 à 12 → Livrable 2

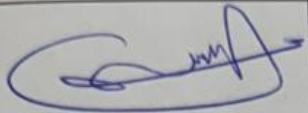
6. Ressources Nécessaires

- Outils de modélisation et de test : Python, Excel, ou autres plateformes équivalentes.
- Documentation sur les méthodes d'optimisation (méthode graphique, Simplexe, dualité).
- Données réelles ou fictives relatives au processus de production laitière.

7. Conclusion

Ce projet vise à illustrer l'importance des méthodes d'optimisation dans l'amélioration des processus. Il offre l'opportunité d'appliquer des outils mathématiques à un cas concret et d'en analyser les impacts opérationnels.

Les résultats attendus permettront une meilleure gestion des ressources et contribueront à une prise de décision plus éclairée.

Signature du chef de projet	Signature du Pr A.REHA	Signature du Pr Y.SAFSOUF
		

Membres de l'équipe :

Aymane EL AZZAOUY — Chef de projet

Imane BENADDI — Membre

Soufiane CHAKIR — Membre

GS-8

REFERENCES

Ouvrages de référence

- [1] Bazaraa, M. S., Jarvis, J. J., & Sherali, H. D. (2010). Linear Programming and Network Flows, 4th ed. John Wiley & Sons. ISBN : 978-0-470-46272-0 .
- [2] Hillier, F. S., & Lieberman, G. J. (2014). Introduction to Operations Research, 10th ed. McGraw-Hill. ISBN : 978-0-07-352345-3.
- [3] Taha, H. A. (2017). Operations Research: An Introduction, 10th ed. Pearson. ISBN : 978-0-13-438706-8.
- [4] Bertsimas, D., & Tsitsiklis, J. N. (1997). Introduction to Linear Optimization. Athena Scientific. ISBN : 978-1-886529-19-9.
- [5] Chvátal, V. (1983). Linear Programming. W. H. Freeman. ISBN : 978-0-7167-1587-0.